

MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

Ngày 14 tháng 6 năm 2026

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
 - Đặt vấn đề & Lý do chọn đề tài
- 2 Kiến Trúc Tập Lệnh (ISA)
 - Tập lệnh cơ bản RV32I
 - Tập lệnh hiện thực trong dự án
- 3 Thiết Kế Hệ Thống (Design)
 - Cấu trúc tổng thể
 - Các khối xử lý logic chức năng
 - Cơ chế Pipeline và điều phối hệ thống
- 4 Chiến Lược Kiểm Thử (Testing)
 - Kế hoạch & Trạng thái kiểm thử

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
 - Đặt vấn đề & Lý do chọn đề tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)
- 4 Chiến Lược Kiểm Thử (Testing)

Xu thế Đa Nhân (Multi-core Processing)

Giới hạn vật lý về tần số xung nhịp buộc kiến trúc máy tính hiện đại phải chuyển dịch sang kiến trúc đa nhân nhằm tối ưu hóa hiệu năng tính toán song song.

Tại sao chọn RISC-V?

- Kiến trúc tập lệnh (ISA) dạng mở, tinh gọn, có tính ứng dụng cao.
- Cấu trúc định dạng lệnh cố định, tạo điều kiện thuận lợi cho việc thiết kế phần cứng bằng ngôn ngữ mô tả phần cứng (Verilog HDL).

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
 - Tập lệnh cơ bản RV32I
 - Tập lệnh hiện thực trong dự án
- 3 Thiết Kế Hệ Thống (Design)
- 4 Chiến Lược Kiểm Thử (Testing)

Tổng quan Tập lệnh RISC-V (RV32I)

RISC-V là gì?

Là một kiến trúc tập lệnh (ISA) mã nguồn mở dựa trên nguyên lý thiết kế máy tính tập lệnh rút gọn (RISC). Cho phép tự do thiết kế, tùy biến và tối ưu hóa phần cứng.

Giải nghĩa bộ lệnh cơ bản RV32I:

- **RV (RISC-V):** Tiêu chuẩn kiến trúc thế hệ thứ 5 từ UC Berkeley.
- **32 (32-bit):** Độ rộng thanh ghi và không gian địa chỉ 32-bit.
- **I (Integer):** Tập lệnh số nguyên cơ bản (gồm 40 lệnh). Đây là tập lệnh bắt buộc phải có cho mọi thiết kế chip RISC-V.

Mục tiêu lựa chọn

Đơn giản hóa thiết kế lõi đơn để tập trung tối ưu bộ điều khiển phân xử (Arbiter) và kết nối đa lõi (8-Core Interconnect).

Đặc trưng kỹ thuật RV32I

Các đặc trưng kỹ thuật:

- Không gian địa chỉ và độ dài lệnh cố định **32-bit**.
- Cơ chế **Load-Store**: Chỉ có lệnh Load/Store mới được truy cập Memory; các phép toán toán học chỉ thực thi trên Tập thanh ghi.
- **32 Thanh ghi đa năng** ($x0 \rightarrow x31$), trong đó $x0$ luôn cố định bằng số 0.
- 6 kiểu định dạng lệnh cơ bản: R, I, S, B, U, J.

Phân loại Tập thanh ghi (Register File)

- $x0$ (**zero**): Luôn bằng 0 (hằng số phần cứng).
- $x1$ (**ra**), $x2$ (**sp**): Địa chỉ trả về & Con trỏ Stack.
- $x3-x4$ (**gp-tp**): Con trỏ toàn cục và con trỏ tiểu trình.
- $x10-x17$ (**a0-a7**): Tham số truyền vào & Kết quả hàm.
- $x5-x7$, $x28-x31$ (**t0-t6**): Thanh ghi tạm thời.
- $x8-x9$, $x18-x27$ (**s0-s11**): Thanh ghi lưu giữ.

6 Kiểu định dạng lệnh cơ bản

	31	25	24	20	19	15	14	12	11	7	6	0		
R-type	funct7				rs2		rs1		funct3		rd		opcode	
I-type	imm[11:0]					rs1		funct3		rd		opcode		
S-type	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
B-type	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U-type	imm[31:12]										rd		opcode	
J-type	imm[20 10:1 11 19:12]										rd		opcode	

Giải thích các trường

- **opcode**: Mã lệnh gốc (7 bit).
- **rd**: Thanh ghi đích.
- **funct3/7**: Mã mở rộng chức năng lệnh.
- **rs1/rs2**: Các thanh ghi nguồn.
- **imm**: Giá trị hằng số nạp trực tiếp.

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
 - Tập lệnh cơ bản RV32I
 - Tập lệnh hiện thực trong dự án
- 3 Thiết Kế Hệ Thống (Design)
- 4 Chiến Lược Kiểm Thử (Testing)

Tập lệnh hiện thực trong dự án

Tập Lệnh Số Nguyên RV32I Subset

40 lệnh cơ bản được hỗ trợ:

- **Tính toán (ALU):** ADD/SUB, SLL/SRL, AND/OR/XOR, SLT (bao gồm cả dạng hằng số Imm như ADDI, ANDI,...).
- **Bộ nhớ:** LW (Load Word), SW (Store Word).
- **Điều khiển luồng:** BEQ, BNE, BLT, BGE, JAL, JALR.
- **Khác:** LUI, AUIPC.

Đồng Bộ Đa Nhân (A-Extension)

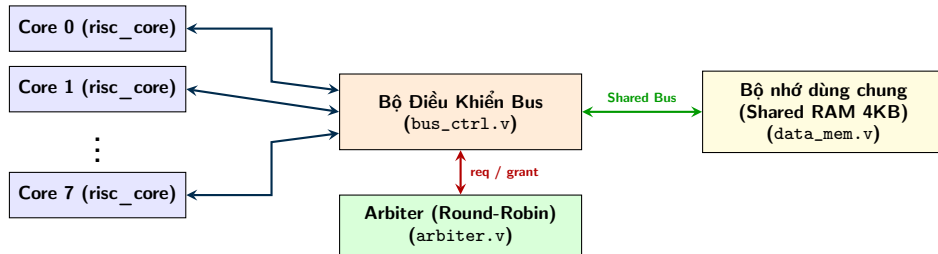
Thiết kế riêng cho hệ thống 8-Core:

- **Độc quyền truy cập:** LR.W (Load-Reserved) và SC.W (Store-Conditional) để xây dựng cấu trúc đồng bộ (Mutex/Semaphore).
- **Phép toán nguyên tử (AMO):** AMOSWAP, AMOADD, AMOAND, AMOOR, AMOXOR, AMOMIN/MAX trực tiếp trên bộ nhớ dùng chung.

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)**
 - **Cấu trúc tổng thể**
 - Các khối xử lý logic chức năng
 - Cơ chế Pipeline và điều phối hệ thống
- 4 Chiến Lược Kiểm Thử (Testing)

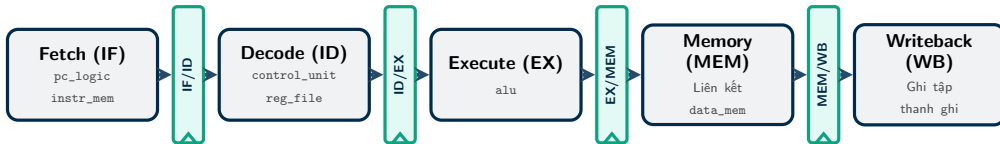
Cấu Trúc Tổng Thể Hệ Thống 8-Core



Hệ thống kết nối đa nhân: **8 lõi CPU** hoạt động song song độc lập, thực hiện phân xử quyền truy cập bộ nhớ thông qua bộ phân xử **Arbiter** và bộ định tuyến bus dữ liệu **Bus Controller**.

Phân bố phần cứng trên 5 tầng Pipeline

Đường truyền dữ liệu (Datapath) ánh xạ trực tiếp các module Verilog vào từng giai đoạn xử lý:



Nguyên lý hoạt động của Đường dữ liệu

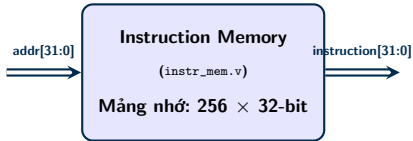
- **Khối logic chức năng** (pc_logic, reg_file, alu,...) thực hiện nhiệm vụ xử lý trong từng tầng riêng biệt.
- **Thanh ghi đệm Pipeline** (IF/ID, ID/EX,...) đóng vai trò chốt dữ liệu giữa các sườn clock, giữ cho lệnh ở các tầng không bị trộn lẫn khi thực thi song song.

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)**
 - Cấu trúc tổng thể
 - Các khối xử lý logic chức năng
 - Cơ chế Pipeline và điều phối hệ thống
- 4 Chiến Lược Kiểm Thử (Testing)

Bộ nhớ lệnh (instr_mem.v)

Sơ đồ khối chức năng



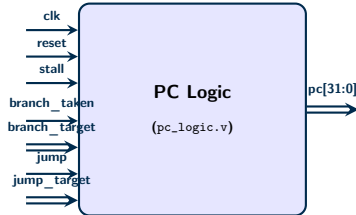
Mã nguồn Verilog đầy đủ

```
module instr_mem #(
    parameter INIT_FILE = "program.hex",
    parameter PROGRAM_WORDS = 256
) (
    input wire [31:0] addr,
    output wire [31:0] instruction
);
    reg [31:0] mem [0:255];
    wire [7:0] word_addr = addr[9:2];

    assign instruction = mem[word_addr];

    integer i;
    initial begin
        // Khởi tạo NOP trước khi nạp
        for (i = 0; i < 256; i = i + 1)
            mem[i] = 32'h0000_0013;
        $readmemh(INIT_FILE, mem, 0, PROGRAM_WORDS - 1);
    end
endmodule
```

Sơ đồ khối chức năng



Nguyên lý cập nhật địa chỉ PC

Quy trình cập nhật PC theo mức độ ưu tiên giảm dần:

- ❶ **Reset:** Đặt địa chỉ PC về 0 khi bắt đầu.
- ❷ **Stall:** Đóng băng PC (giữ nguyên giá trị cũ) khi gặp xung đột dữ liệu hoặc tài nguyên.
- ❸ **Jump:** Nhập địa chỉ nhảy không điều kiện ('jump_target') từ các lệnh 'JAL', 'JALR'.
- ❹ **Branch:** Nhập địa chỉ rẽ nhánh ('branch_target') khi điều kiện rẽ nhánh thỏa mãn (branch_taken = 1).

PC Logic: Hiện thực mã nguồn Verilog

Khai báo Module & Ports

```
module pc_logic (  
    input wire      clk,  
    input wire      reset,  
    input wire      stall,  
  
    input wire      branch_taken,  
    input wire [31:0] branch_target,  
  
    input wire      jump,  
    input wire [31:0] jump_target,  
  
    output reg [31:0] pc  
);
```

Khối Logic Cập Nhật PC

```
always @(posedge clk or posedge reset) begin  
    if (reset) begin  
        pc <= 32'd0;  
    end  
    else if (stall) begin  
        pc <= pc;  
    end  
    else if (jump) begin  
        pc <= jump_target;  
    end  
    else if (branch_taken) begin  
        pc <= branch_target;  
    end  
    else begin  
        pc <= pc + 32'd4;  
    end  
end  
  
endmodule
```

PC Logic: Thiết lập môi trường Testbench

Khai báo Tín hiệu & Mạch nạp

```
'timescale 1ns/1ps

module pc_logic_tb;

    reg clk;
    reg reset;
    reg stall;

    reg branch_taken;
    reg [31:0] branch_target;

    reg jump;
    reg [31:0] jump_target;

    wire [31:0] pc;
```

Khởi tạo UUT & Bộ tạo Xung nhịp

```
pc_logic uut (
    .clk(clk),
    .reset(reset),
    .stall(stall),
    .branch_taken(branch_taken),
    .branch_target(branch_target),
    .jump(jump),
    .jump_target(jump_target),
    .pc(pc)
);

always #5 clk = ~clk;
```

PC Logic: Kịch bản kiểm thử

Kịch bản kích thích chính

```
initial begin
    clk = 0; reset = 1; stall = 0;
    branch_taken = 0; branch_target = 0;
    jump = 0; jump_target = 0;
    #10 reset = 0;
    #20; // Normal PC increment
    stall = 1; #10; stall = 0; // Stall
    branch_taken = 1; branch_target = 100;
    #10; branch_taken = 0; // Branch
    jump = 1; jump_target = 200;
    #10; jump = 0; // Jump
    #20; $finish;
end
```

Giám sát tín hiệu & Lưu dạng sóng

```
initial begin
    $monitor(
        "t=%0t pc=%0d s=%b b=%b j=%b",
        $time, pc, stall, branch_taken, jump
    );
end

initial begin
    $dumpfile("pc_logic_tb.vcd");
    $dumpvars(0, pc_logic_tb);
end

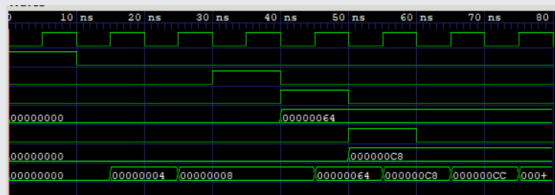
endmodule
```

PC Logic: Kết quả mô phỏng & Giải đồ sóng

Console Output (\$monitor)

```
Time=0 | PC=      0 | stall=0 | branch=0 | jump=0
Time=15000 | PC=     4 | stall=0 | branch=0 | jump=0
Time=25000 | PC=     8 | stall=0 | branch=0 | jump=0
Time=30000 | PC=     8 | stall=1 | branch=0 | jump=0
Time=40000 | PC=     8 | stall=0 | branch=1 | jump=0
Time=45000 | PC=    100 | stall=0 | branch=1 | jump=0
Time=50000 | PC=    100 | stall=0 | branch=0 | jump=1
Time=55000 | PC=    200 | stall=0 | branch=0 | jump=1
Time=60000 | PC=    200 | stall=0 | branch=0 | jump=0
Time=65000 | PC=    204 | stall=0 | branch=0 | jump=0
Time=75000 | PC=    208 | stall=0 | branch=0 | jump=0
tb/pc_logic_tb.v:63: $finish called at 80000 (1ps)
```

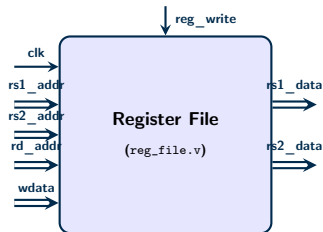
Giải đồ sóng (Waveform)



- **Stall (30-40ns):** PC giữ nguyên giá trị cũ (= 8).
- **Branch (40-50ns):** PC nhảy tới '0x64' (= 100).
- **Jump (50-60ns):** PC nhảy tới '0xC8' (= 200).

Khối Tập Thanh Ghi (reg_file.v)

Sơ đồ khối chức năng



Nguyên lý hoạt động

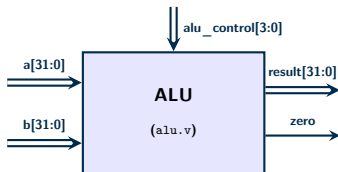
- Quy mô: 32 thanh ghi 32-bit (x0 cứng về 0).
- Đọc bất đồng bộ: rs1_data, rs2_data.
- Ghi đồng bộ: rd khi reg_write=1.
- Cầu bypass: Tránh trễ 1 chu kỳ khi đọc-ghi cùng thanh ghi.

Mã nguồn Verilog tập thanh ghi

```
module reg_file (  
    input wire      clk, reset,  
    input wire [4:0] rs1_addr, rs2_addr, rd_addr,  
    input wire [31:0] write_data,  
    input wire      reg_write,  
    output wire [31:0] rs1_data, rs2_data  
);  
    reg [31:0] registers [0:31];  
  
    assign rs1_data = (rs1_addr == 5'd0) ? 32'd0 :  
                      (reg_write && (rd_addr == rs1_addr))  
                      ? write_data : registers[rs1_addr];  
    assign rs2_data = (rs2_addr == 5'd0) ? 32'd0 :  
                      (reg_write && (rd_addr == rs2_addr))  
                      ? write_data : registers[rs2_addr];  
  
    integer i;  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            for (i = 0; i < 32; i = i + 1)  
                registers[i] <= 32'd0;  
        end else if (reg_write && (rd_addr != 5'd0)) begin  
            registers[rd_addr] <= write_data;  
        end  
    end  
endmodule
```

Khối Tính Toán Số Học & Logic (alu.v)

Sơ đồ khối chức năng



Đặc tả hoạt động

- **Phép toán số học:** ADD (cộng), SUB (trừ), so sánh có dấu SLT.
- **Phép toán logic:** AND, OR, XOR và dịch chuyển bit logic (SLL, SRL).
- **Cờ Zero:** Tín hiệu ra 1-bit, tích cực mức cao khi kết quả toán học bằng 0 (dùng cho lệnh rẽ nhánh).

Hiện thực khối Case ALU

```
always @(*) begin
    case (alu_control)
        4'b0010: result = a + b; // ADD
        4'b0110: result = a - b; // SUB
        4'b0000: result = a & b; // AND
        4'b0001: result = a | b; // OR
        4'b0100: result = a ^ b; // XOR
        4'b0011: result = a << b[4:0]; // SLL
        4'b0101: result = a >> b[4:0]; // SRL
        4'b0111: result = ($signed(a) < $signed(b))
                        ? 32'd1 : 32'd0; // SLT
        default: result = 32'b0;
    endcase
end

// Co bao Zero
assign zero = (result == 32'b0);
```

ALU: Kịch bản kiểm thử (Testbench)

DUT & Khai báo tín hiệu

```
module alu_tb();
    reg [31:0] a, b;
    reg [3:0] alu_control;
    wire [31:0] result;
    wire zero;

    // Khoi tao Device Under Test (DUT)
    alu dut (
        .a(a), .b(b),
        .alu_control(alu_control),
        .result(result), .zero(zero)
    );

    localparam AND = 4'b0000;
    localparam OR = 4'b0001;
    localparam ADD = 4'b0010;
    localparam SUB = 4'b0110;
    localparam SLT = 4'b0111;
```

Kịch bản kích thích chính

```
initial begin
    $dumpfile("tb/alu_tb.vcd");
    $dumpvars(0, alu_tb);

    // Case 1: Phep cong (ADD)
    a = 32'd10; b = 32'd20; alu_control = ADD; #10;

    // Case 2: Phep tru (SUB) -> zero = 1
    a = 32'd50; b = 32'd50; alu_control = SUB; #10;

    // Case 3: Phep logic AND
    a = 32'hAAAA_AAAA; b = 32'h5555_5555;
    alu_control = AND; #10;

    // Case 4: So sanh nho hon SLT (co dau)
    a = -32'd10; b = 32'd5; alu_control = SLT; #10;

    $finish;
end
```

ALU: Kết quả mô phỏng Console

Mô phỏng thực thi chính xác các phép tính và kiểm chứng cờ rẽ nhánh:

Kết quả xuất ra màn hình (Console Output)

Time	A (Hex) [Dec]	B (Hex) [Dec]	Control	Result [Dec]	Zero
10	0x0000000a (10)	0x00000014 (20)	ADD	0x0000001e (30)	0
20	0x00000032 (50)	0x00000032 (50)	SUB	0x00000000 (0)	1
30	0xaaaaaaaa (-1.43G)	0x55555555 (1.43G)	AND	0x00000000 (0)	1
40	0x0000000f (15)	0x00000019 (25)	SLT	0x00000001 (1)	0
50	0xffffffff6 (-10)	0x00000005 (5)	SLT	0x00000001 (1)	0

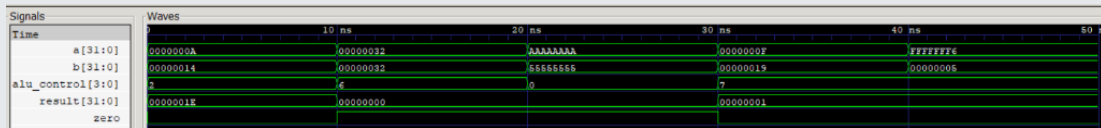
Kiểm tra kết thúc.

Nhận xét kết quả:

- **Tính toán chính xác:** Phép 'ADD' ($10 + 20 = 30$) và phép 'SUB' ($50 - 50 = 0$) cho kết quả khớp lý thuyết.
- **Cờ Zero:** Tự động kích hoạt lên '1' khi kết quả của phép 'SUB' và 'AND' bằng 0.
- **Phép toán có dấu SLT:** So sánh -10 và 5 cho kết quả đúng bằng '1' (SLT thực hiện so sánh có dấu \$signed).

ALU: Giải Đồ Sóng Mô Phỏng (GTKWave)

Giải đồ sóng chi tiết thu được từ bộ mô phỏng



• Tín hiệu ngõ vào (Inputs)

- a[31:0], b[31:0]: Toán hạng ngõ vào (dạng Hex).
- alu_control[3:0]: Mã phép toán điều khiển (2=ADD, 6=SUB, 0=AND, 7=SLT).

• Tín hiệu ngõ ra (Outputs)

- result[31:0]: Kết quả tính toán (dạng Hex).
- zero: Cờ báo bằng không, mức cao (1) khi result = 0.

ALU: Phân Tích Chi Tiết Sóng Mô Phỏng

Phân tích 5 Test Case thực thi

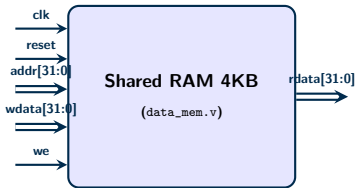
- **0ns - 10ns (ADD):** $10 + 20 = 30$ (0x1E). Kết quả khác 0 $\rightarrow$ cờ zero = 0.
- **10ns - 20ns (SUB):** $50 - 50 = 0$. Kết quả bằng 0 $\rightarrow$ cờ zero = 1 kéo lên mức cao.
- **20ns - 30ns (AND):** $0xFFFFFFFF \text{ AND } 0x55555555 = 0$. Cờ zero tiếp tục giữ ở mức cao.
- **30ns - 40ns (SLT - Số dương):** So sánh $15 < 25$ (Đúng $\rightarrow$ result = 1). Cờ zero = 0.
- **40ns - 50ns (SLT - Số âm):** So sánh có dấu -10 (0xFFFFF6) < 5 (Đúng $\rightarrow$ result = 1).

Nhận xét kỹ thuật

- Phép so sánh SLT hoạt động chính xác với số âm (\$signed) nhờ biểu diễn bù 2, không bị nhầm lẫn với số không dấu.
- Cờ zero phản hồi tức thời theo giá trị của result.
- Thiết kế ALU hoạt động đúng chức năng của tập lệnh RV32I.

Bộ Nhớ Dữ Liệu Dùng Chung (data_mem.v)

Sơ đồ khối chức năng



Cơ chế hoạt động bộ nhớ

- **Căn lề địa chỉ (Word-aligned):** Lấy 10 bit địa chỉ thực tế `addr[11:2]` để lập chỉ mục 1024 từ nhớ (mỗi từ 32-bit).
- **Đọc tổ hợp (Bất đồng bộ):** Trả về `rdata` lập tức khi đổi địa chỉ và `re=1` (thuận tiện cho kết nối Bus).
- **Ghi đồng bộ:** Cập nhật mảng nhớ tại cạnh lên `clk` khi `we=1`.

Mã nguồn RAM 4KB dùng chung

```
module data_mem (  
    input wire          clk, reset,  
    input wire [31:0]   addr, wdata,  
    input wire          we, re,  
    output reg  [31:0]  rdata  
);  
  
    reg [31:0] mem [0:1023];  
    wire [9:0] word_addr = addr[11:2];  
  
    always @(*) begin  
        if (re) rdata = mem[word_addr];  
        else   rdata = 32'd0;  
    end  
  
    integer i;  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            for (i = 0; i < 1024; i = i + 1)  
                mem[i] <= 32'd0;  
        end else if (we) begin  
            mem[word_addr] <= wdata;  
        end  
    end  
endmodule
```

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)**
 - Cấu trúc tổng thể
 - Các khối xử lý logic chức năng
 - Cơ chế Pipeline và điều phối hệ thống
- 4 Chiến Lược Kiểm Thử (Testing)

Cơ chế hoạt động của Pipeline Registers

Các thanh ghi đệm hoạt động như các "vách ngăn" đồng bộ bằng xung nhịp Clock:

Đồng bộ hóa & Dịch chuyển

- **Lưu giữ trạng thái:** Đảm bảo dữ liệu của lệnh trước không bị đè bởi lệnh sau khi đang xử lý song song.
- **Xếp hàng tín hiệu:** Tín hiệu điều khiển từ bộ Decode (`control_unit`) được đẩy dọc theo các thanh ghi đệm (`id_ex_reg_write`, `ex_mem_mem_read`,...) để thực thi đúng thời điểm.

Đường đi tắt (Forwarding Paths)

Bao vây các thanh ghi đệm để giải quyết xung đột dữ liệu:

- Bơm kết quả trực tiếp từ ngõ ra ALU (`ex_mem_result`) hoặc dữ liệu ghi ngược (`mem_wb_write_data`).
- Đưa thẳng về ngõ vào ALU tầng **EX** thông qua các bộ MUX lớn, giảm thiểu số chu kỳ bị Stall.

Bộ Giải Mã Lệnh (control_unit.v)

Bộ điều khiển control_unit.v là bộ giải mã trung tâm (decoder) hoạt động tại tầng **Decode (ID)**:

Giải mã tín hiệu điều khiển

Từ mã lệnh 32-bit, CU trích xuất opcode, funct3, funct7 để sinh:

- **Đường truyền dữ liệu:** reg_write, alu_src, mem_to_reg.
- **Điều khiển ALU:** alu_control[3:0] (ADD, SUB, SLL, SRL, AND, OR, XOR, SLT).
- **Rẽ nhánh & Nhảy:** branch, branch_type, jump, jalr.

Tín hiệu đa nhân đặc biệt

Hỗ trợ các tín hiệu điều khiển truy cập bộ nhớ và đồng bộ:

- mem_lr / mem_sc: Lệnh nguyên tử Load-Reserved / Store-Conditional.
- mem_amo: Kích hoạt phép toán nguyên tử trực tiếp trên bộ nhớ.
- amo_op[3:0]: Chọn phép toán AMO (swap, add, logic, min/max).

Bộ tạo hằng số (Immediate Generator): Giải mã và căn lề dấu (sign-extend) hằng số imm cho các định dạng lệnh I, S, B, U, J.

Quản lý xung đột trong Pipeline (hazard_unit)

Quá trình chạy gộp đầu lệnh (Pipelining) sinh ra các xung đột dữ liệu và điều khiển, được xử lý trực tiếp bên trong `risc_core.v`:

1. Xung đột dữ liệu (Data Hazard)

Xảy ra khi lệnh sau cần đọc giá trị thanh ghi mà lệnh trước chưa ghi xong.

- **Mạch Forwarding (Bơm tắt):** Chuyển kết quả trực tiếp từ ngõ ra EX/MEM hoặc MEM/WB về ngõ vào ALU mà không cần chờ Write Back.
- **Load-use Stall:** Với lệnh Load, kích hoạt tín hiệu `load_use_stall` để hoãn Pipeline 1 chu kỳ khi lệnh kế sau sử dụng kết quả Load.

2. Xung đột điều khiển (Control)

Xảy ra khi các lệnh nhảy hoặc rẽ nhánh thay đổi dòng chảy địa chỉ PC.

- Khi tín hiệu `redirect_taken` kích hoạt (do Branch hoặc Jump).
- **Cơ chế Flush (Xóa):** Reset các thanh ghi đệm tầng nạp sai lệnh (`if_id_valid <= 0`, `id_ex_valid <= 0`), đưa các tín hiệu điều khiển về 0 (tạo lệnh rỗng NOP).

Trục Bus & Phân Xử Đa Nhân (arbiter.v & bus_ctrl.v)

Để liên kết **8 lõi CPU** dùng chung RAM 4KB, hệ thống sử dụng cơ chế bus điều phối nguyên tử:

- **Bộ phân xử Arbiter (arbiter.v) - Thuật toán Round-Robin:**

- Quét 8 Core từ con trở ưu tiên (priority_ptr). Core đầu tiên gửi yêu cầu (request) sẽ được cấp quyền.
- **Tín hiệu cấp quyền (grant):** Chỉ kích hoạt trong **1 chu kỳ xung nhịp (pulse)**, sau đó tự xóa. Con trở ưu tiên dịch chuyển tiếp sang nhân bên cạnh ($\text{priority_ptr} \leq \text{winner} + 1$) nhằm chia sẻ băng thông công bằng, tránh "đói" tài nguyên.
- Sử dụng mặt nạ $\text{new_request} = \text{request} \ \& \ \sim \text{last_granted}$ để ngăn chặn 1 Core liên tục chiếm giữ Bus.

- **Bộ điều khiển Bus (bus_ctrl.v) & Hỗ trợ Nguyên tử (Atomic):**

- Định tuyến song song địa chỉ và dữ liệu giữa Core được cấp quyền (grant) và RAM.
- **Xử lý khóa nguyên tử (Atomic Locks):** Lưu vết địa chỉ đặt khóa cho các lệnh LR.W/SC.W. Nếu xảy ra xung đột ghi từ Core khác, khóa bị xóa và lệnh SC.W trả về thất bại (kết quả 1).

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)
- 4 Chiến Lược Kiểm Thử (Testing)
 - Kế hoạch & Trạng thái kiểm thử

Chiến Lược Kiểm Thử & Đọc Giản Đồ Sóng

Hệ thống được kiểm thử bài bản từ cấp độ Module cô lập đến chạy ứng dụng thực tế:

Cấp độ kiểm thử	Cách thực hiện kiểm chứng trên Waveform	Trạng thái
1. Single Core	Nạp file Hex mẫu. Xem PC tăng tiến và các toán hạng ALU khớp.	Hoàn thành
2. Pipeline Hazard	Gây lỗi đọc-ghi liên tiếp. Kiểm tra tín hiệu ForwardA/B bật lên.	Hoàn thành
3. Multi-Core	Cho 8 Core cùng chạy, theo dõi tín hiệu phân phối grant của Arbiter.	Đang triển khai

Minh chứng Thuyết trình (Bằng chứng trực quan)

Trong báo cáo chi tiết, chúng em sử dụng ảnh chụp Waveform từ công cụ mô phỏng để giải thích rõ ràng các pha **Cache Miss / Stall** và cách hệ thống tự động sửa sai trên dòng xung nhịp thực tế.